



# Python Coursework

---

For each logical operation described in the tasks:

- Implement a separate function.
- Use descriptive function names that reflect the task.
- Each function must return the result.
- The main block (`if __name__ == "__main__":`) should call the functions and print results clearly.
- Do not use external libraries unless necessary
- **Do NOT hardcode predefined values inside the program.**
- Use `input()` to collect data dynamically from the user.
- The program must work for different inputs without modifying the source code.

## Task 1: List Operations in Python

### Problem

---

Write a Python program named: `task1_lists.py` that performs the following operations on lists.

### Part A: Integer List Operations

Given a list of integers, implement logic to:

1. Multiply all the items in the list.
2. Find the largest number in the list.
3. Find the smallest number in the list.
4. Remove duplicate elements while preserving the original order.

5. Check whether the list is empty.
6. Find the largest odd number in the list. If no odd numbers exist, print: "No odd numbers found"
7. Remove elements at index positions 0, 4, and 5 from the list. Remove only if the index exists. The program must not crash for short lists.

## **Part B: Tuple List Sorting**

8. Given a list of non-empty tuples, Sort the list in increasing order based on the last element of each tuple.

Example:

Input: [(1, 3), (2, 1), (4, 2)]

Output: [(2, 1), (4, 2), (1, 3)]

## **Part C: Word List Analysis**

9. Given a list of words, Count the total number of lowercase letters across all words in the list.

## **Part D: Consecutive Element Extraction**

10. Given a list of integers and a positive integer k, extract elements that appear exactly k times consecutively. If an element appears more than or fewer than k times consecutively, do NOT include it.

**Example:**

Input : [1, 1, 3, 4, 4, 5, 6, 7]

k = 2

Output: [1, 4]

---

## Task 2: Tuple Operations in Python

### Problem

---

Write a Python program named: task2\_tuples.py that performs the following tuple operations.

### Part A: Tuple Creation and Access

1. Create a tuple containing elements of different data types (e.g., integer, float, string, boolean).
2. Create a tuple containing at least five numbers and Print one specific element (choose any valid index).
3. Retrieve the 4th element from the end of a given tuple. If the tuple contains fewer than 4 elements, display an appropriate message. The program must not crash.

### Part B: Tuple Modification

4. Add an item to an existing tuple. Since tuples are immutable, demonstrate the correct method to achieve this. Do not modify the original tuple directly.

### Part C: Tuple Conversion

5. Convert a tuple into a dictionary. If the tuple contains key-value pairs (e.g., ( "a",1), ("b",2) )), convert directly. If the tuple is a simple sequence of values, convert it into a dictionary where: Keys are the index positions, Values are the tuple elements. The program must clearly handle one of the given approaches.

### Part D: Tuple Transformation in a List

6. Given a list of tuples: [(10, 20, 40), (40, 50, 60), (70, 80, 90)], Replace the last element of each tuple with 100.

Expected Output: [(10, 20, 100), (40, 50, 100), (70, 80, 100)]

**Requirements:**

- Do not modify tuples directly (since they are immutable).
- Create updated tuples properly.
- The program must work for tuples of any length (greater than or equal to 1).

---

## **Task 3: Dictionary Operations in Python**

### **Problem**

---

Write a single Python program named: `task3_dictionary.py` that performs the following dictionary operations.

### **Part A: Sorting a Dictionary by Value**

1. Given a dictionary with numeric values: Sort the dictionary in ascending order and descending order by value. Display both results clearly.

### **Part B: Iterating Through a Dictionary**

2. Demonstrate how to iterate through a dictionary using for loops:
  - Iterate over keys only.
  - Iterate over values only.
  - Iterate over key-value pairs. Display the results clearly.

### **Part C: Merging Dictionaries**

3. Merge two dictionaries into a single dictionary.

**Requirements:**

- If duplicate keys exist, the value from the second dictionary should overwrite the first.

- The program must not crash if one or both dictionaries are empty.

## **Part D: Aggregation Operations**

4. Calculate the sum of all numeric values in a dictionary.
5. Calculate the product (multiplication) of all numeric values in a dictionary.

### **Requirements:**

- Assume values are numeric.
- Handle the case of an empty dictionary appropriately.

## **Part E: Sorting by Key**

6. Sort a dictionary in ascending order by key. Display the sorted result clearly.

## **Part F: Removing Duplicates**

7. Remove duplicate values from a dictionary.

### **Clarification:**

- Since dictionary keys must be unique, “duplicate” refers to duplicate values.
- If multiple keys share the same value, keep only the first occurrence and remove the others.

### **Example:**

Input: {'a': 10, 'b': 20, 'c': 10}

Output: {'a': 10, 'b': 20}